

Chapter 22. Troubleshooting Clusters

When a Kubernetes cluster experiences infrastructure-level failures, the impact can be catastrophic. Pods refuse to schedule, applications become unreachable, and entire nodes disappear from the cluster, potentially affecting hundreds of workloads simultaneously. Unlike application-specific issues that might affect a single service, problems with cluster components and nodes strike at the very foundation of your Kubernetes environment, making their swift diagnosis and resolution critical for maintaining system availability.

This chapter equips you with the skills to troubleshoot these infrastructure-level problems. You'll learn to decode node conditions, investigate why nodes transition to `NotReady` states, resolve resource pressure situations, and trace Pod scheduling failures back to their root causes.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objectives:

- Troubleshoot clusters and nodes
 - Troubleshoot cluster components
-

Inspecting the Status of Cluster Nodes

There are many influencing factors that can render a Kubernetes cluster faulty on the component level. It's a good idea to list the nodes available in the cluster to identify potential issues:

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
control-plane-node	Ready	control-plane	2m45s	v1.33.2
worker-node-1	Ready	<none>	2m36s	v1.33.2
worker-node-2	Ready	<none>	2m29s	v1.33.2
worker-node-3	Ready	<none>	2m22s	v1.33.2

The output will give you a lay of the land. You can easily identify the responsibility of each node from the `ROLES` column, the Kubernetes ver-

sion used, and the current health status.

There are a couple of things to look out for when identifying issues at a high level:

- Is the health status for the node anything other than `Ready` ?
- Does the version of a node deviate from the version of other nodes?

In the following sections you can find individual sections on troubleshooting control plane nodes versus worker nodes.

Inspecting the Status of Cluster Components

Among those [components available on the control plane node](#) are the following:

kube-apiserver

Exposes the Kubernetes API used by clients like `kubectl` for managing objects

etcd

A key-value store for storing the cluster data

kube-scheduler

Selects nodes for Pods that have been scheduled but not created

core-dns

Serves as the cluster's DNS server, automatically creating DNS records that allow Pods and Services to discover each other by name rather than IP addresses, while also handling external DNS resolution for workloads

kube-controller-manager

Runs controller processes (e.g., the job controller responsible for Job object execution)

cloud-controller-manager (optional)

Links cloud provider-specific APIs to the Kubernetes cluster. This controller is not available in on-premise cluster installations of Kubernetes

In addition to the components running specifically on the control plane nodes, every node (control plane and worker nodes) can contain the following components:

kubelet

Ensures that all Pods are running, including their containers

kube-proxy (optional)

Maintains the network rules on nodes to implement Services

Container runtime

The software component responsible for running containers

To discover those components and their status, list the Pods available in the namespace `kube-system`. Be aware that some components do not run in a Pod, e.g., the kubelet or the container runtime. Here, you can find the list of components:

```
$ kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
etcd	1/1	Running	1 (11d ago)	29d
kube-apiserver	1/1	Running	1 (11d ago)	29d
kube-controller-manager	1/1	Running	1 (11d ago)	29d
kube-scheduler	1/1	Running	1 (11d ago)	29d
...				

Any status that does not show `Running` should be inspected further. You can retrieve the logs for control plane component Pods in the same fashion you do for any other Pod, using the `logs` command. The following command downloads the logs for the kube-apiserver component:

```
$ kubectl logs kube-apiserver -n kube-system
```

In managed Kubernetes environments (EKS, GKE, AKS), many of these control plane Pods won't be visible, as they run on provider-managed infrastructure outside your view. You might only see add-on Pods like kube-proxy, CoreDNS, and CNI components. While the CKA exam uses self-managed clusters where all control plane Pods are visible and accessible, it's important to understand this distinction for real-world scenarios where troubleshooting control plane issues in managed services requires different approaches and often involves the cloud provider's support.

Troubleshooting Node Issues

Control plane nodes are the critical components for keeping a cluster operational. As described in [“Managing a Highly Available Cluster”](#), a cluster can consist of more than one control plane node to ensure a high degree of uptime. Detecting that one of the control plane nodes is faulty should be treated with extreme urgency to avoid compromising high-availability characteristics. For more information on troubleshooting techniques and root-cause analysis, reference the [Kubernetes documentation](#).

Rendering Cluster Information

To further diagnose issues on the control plane node, run the command `kubectl cluster-info`. As you can see in the following output, the command renders the addresses of the control plane and other cluster services:

```
$ kubectl cluster-info
```

```
Kubernetes control plane is running at https://192.168.64.21:8443
CoreDNS is running at https://192.168.64.21:8443/api/v1/namespaces/ \
kube-system/services/kube-dns:dns/proxy
```

To further debug and diagnose cluster problems, use `kubectl cluster-info o`

For a detailed view of the cluster logs, append the `dump` subcommand. Due to the pages and pages of log messages, I won’t render the output in this book. Parse through the message to see if you can find any errors:

```
$ kubectl cluster-info dump
```

Node Showing NotReady Status

Worker nodes are responsible for managing the workload. Make sure you have a sufficient number of worker nodes available to distribute the load. For a deeper discussion on how to join worker nodes to a cluster, see [Chapter 4](#).

Any of the nodes available in a cluster can transition into an error state. It’s your job as a Kubernetes administrator to identify those situations and fix them in a timely manner. When listing the nodes of a cluster, you may see that a worker node is not in the `Ready` state, which is a good indicator that it’s not available to handle the workload. In the output of the `get nodes` command, you can see that the node named `worker-1` is in the `NotReady` state:

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
control-plane	Ready	control-plane	4d20h	v1.33.2
worker-1	NotReady	<none>	4d20h	v1.33.2
worker-2	Ready	<none>	4d20h	v1.33.2

The `NotReady` state means that the node is unused and will accumulate operational costs without actually scheduling workload. There might be a variety of reasons why the node entered this state. The following list shows the most common reasons:

Insufficient resources

The node may be low on memory or disk space.

Issues with the kubelet process

The process may have crashed or stopped on the node. Therefore, it cannot communicate with the API server running on any of the control plane nodes anymore.

Issues with kube-proxy

The Pod running kube-proxy is responsible for network communication from within the cluster and from the outside. The Pod transitioned into a nonfunctional state.

SSH into the relevant worker node(s) and start your investigation.

Checking Available Resources

A good way to identify the root cause of an unavailable worker node is to look at its details. The `describe node` command renders the section labeled `Conditions`:

```
$ kubectl describe node worker-1
```

```
....
```

```
Conditions:
```

Type	Status	LastHeartbeatTime	LastTransitionTime	Reason	Message
NetworkUnavailable	False	Thu, 20 Jan 2022 18:12:13 +0000	Thu, 20 Jan 2022 18:12:13 +0000	CalicoIsUp	Calico is running on this node
MemoryPressure	False	Tue, 25 Jan 2022 15:59:18 +0000	Thu, 20 Jan 2022 18:11:47 +0000	KubeletHasSufficientMemory	kubelet has sufficient memory available
DiskPressure	False	Tue, 25 Jan 2022 15:59:18 +0000			

```

Thu, 20 Jan 2022 18:11:47 +0000    KubeletHasNoDiskPressure \
    kubelet has no disk pressure
PIDPressure            False      Tue, 25 Jan 2022 15:59:18 +0000 \
    Thu, 20 Jan 2022 18:11:47 +0000    KubeletHasSufficientPID \
    kubelet has sufficient PID available
Ready                  True       Tue, 25 Jan 2022 15:59:18 +0000 \
    Thu, 20 Jan 2022 18:12:07 +0000    KubeletReady          \
    kubelet is posting ready status. AppArmor enabled
...

```

The table contains information about the resources available to the node, as well as an indication of other services like networking. See if any of the resource types render the status `True` or `Unknown`, which means that there's an issue with the particular resource. You can further troubleshoot unavailable resources with a system-level command.

To check on memory and the number of processes running, use the `top` command:

```

$ top
top - 18:45:09 up 1 day,  2:21,  1 user,  load average: 0.13, 0.13, 0.15
Tasks: 116 total,   3 running,  70 sleeping,   0 stopped,   0 zombie
%Cpu(s):  1.5 us,   0.8 sy,   0.0 ni, 97.7 id,   0.0 wa,   0.0 hi,   0.0 si,   0
KiB Mem : 1008552 total,  134660 free,  264604 used,  609288 buff/cach
KiB Swap:          0 total,          0 free,          0 used.  611248 avail Mem
...

```

To check on the available disk space, use the command `df` :

```

$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            480M     0  480M   0% /dev
tmpfs           99M    1.0M   98M   2% /run
/dev/sda1       39G   2.7G   37G   7% /
tmpfs           493M     0  493M   0% /dev/shm
tmpfs           5.0M     0   5.0M   0% /run/lock
tmpfs           493M     0  493M   0% /sys/fs/cgroup
vagrant         1.9T  252G   1.6T  14% /vagrant
tmpfs           99M     0   99M   0% /run/user/1000

```

Checking the kubelet Process

Some conditions rendered by the `describe node` command mention the kubelet process. If you look at the `Message` column, you might get an idea if the kubelet process is running properly. To troubleshoot a misbehaving kubelet process, run the following `systemctl` command:

```
$ systemctl status kubelet
```

```
● kubelet.service - kubelet: The Kubernetes Node Agent
   Loaded: loaded (/lib/systemd/system/kubelet.service; enabled; \
   vendor preset: enabled)
   Drop-In: /etc/systemd/system/kubelet.service.d
            └─10-kubeadm.conf
   Active: active (running) since Thu 2022-01-20 18:11:41 UTC; 5 days ago
     Docs: https://kubernetes.io/docs/home/
  Main PID: 6537 (kubelet)
    Tasks: 15 (limit: 1151)
   CGroup: /system.slice/kubelet.service
            └─6537 /usr/bin/kubelet \
               --bootstrap-kubeconfig=/etc/kubernetes/bootstrap-kubelet.conf \
               --kubeconfig=/etc/kubernetes/kubelet.conf \
               --config=/var/lib/kubelet/config.yaml --network-lines 1-10/10
```

The most important information in the output is the value of the `Active` attribute. If it says something other than `active (running)`, then you will need to dig deeper. Use `journalctl` to take a look at the log files of the process:

```
$ journalctl -u kubelet.service
```

```
-- Logs begin at Thu 2022-01-20 18:10:41 UTC, end at
Tue 2022-01-25 18:44:05 UTC. --
Jan 20 18:11:31 worker-1 systemd[1]: Started kubelet: The Kubernetes Node
Jan 20 18:11:31 worker-1 systemd[1]: kubelet.service: Current command vani
from the unit file, execution of the command list won't be resumed.
Jan 20 18:11:31 worker-1 systemd[1]: Stopping kubelet: The Kubernetes
Node Agent...
Jan 20 18:11:31 worker-1 systemd[1]: Stopped kubelet: The Kubernetes Node
Jan 20 18:11:31 worker-1 systemd[1]: Started kubelet: The Kubernetes Node
....
```

You will want to restart the process once you have identified the issue in the logs and fixed it:

```
$ systemctl restart kubelet
```

Checking Certificate Validity

Sometimes, the certificate used by the kubelet can expire. Make sure that the values for the attributes `Issuer` and `Not After` are correct:

```
$ openssl x509 -in /var/lib/kubelet/pki/kubelet.crt -text
```

```
Certificate:
```

```
    Data:
```

```
        Version: 3 (0x2)
```

```

Serial Number: 2 (0x2)
Signature Algorithm: sha256WithRSAEncryption
Issuer: CN = worker-1-ca@1642702301
Validity
    Not Before: Jan 20 17:11:41 2022 GMT
    Not After : Jan 20 17:11:41 2023 GMT
Subject: CN = worker-1@1642702301
...

```

For a quick rundown of all certificates in the cluster's Public Key Infrastructure (PKI), you can use the following command:

\$ kubectl get csr

```

[check-expiration] Reading configuration from the cluster...
[check-expiration] FYI: You can look at this config file with 'kubectl -n
kube-system get cm kubeadm-config -o yaml'

```

CERTIFICATE	EXPIRES	RESIDUAL TIME	...
admin.conf	Aug 31, 2026 14:28 UTC	364d	...
apiserver	Aug 31, 2026 14:28 UTC	364d	...
apiserver-etcd-client	Aug 31, 2026 14:28 UTC	364d	...
apiserver-kubelet-client	Aug 31, 2026 14:28 UTC	364d	...
controller-manager.conf	Aug 31, 2026 14:28 UTC	364d	...
etcd-healthcheck-client	Aug 31, 2026 14:28 UTC	364d	...
etcd-peer	Aug 31, 2026 14:28 UTC	364d	...
etcd-server	Aug 31, 2026 14:28 UTC	364d	...
front-proxy-client	Aug 31, 2026 14:28 UTC	364d	...
scheduler.conf	Aug 31, 2026 14:28 UTC	364d	...
super-admin.conf	Aug 31, 2026 14:28 UTC	364d	...

CERTIFICATE AUTHORITY	EXPIRES	RESIDUAL TIME	...
ca	Aug 29, 2035 14:28 UTC	9y	...
etcd-ca	Aug 29, 2035 14:28 UTC	9y	...
front-proxy-ca	Aug 29, 2035 14:28 UTC	9y	...

You can renew all certificates necessary to run the control plane with the following command:

\$ kubectl get csr

You must restart the kube-apiserver, kube-controller-manager, kube-scheduler, and etcd, so that they can use the new certificates.

Checking the kube-proxy Pod

The kube-proxy components run in a set of dedicated Pods in the namespace `kube-system`. You can clearly identify the Pods by their naming

prefix `kube-proxy` and the appended hash. Verify if any of the Pods states a different status than `Running`. Each of the `kube-proxy` Pods runs on a dedicated worker node. You can add the `-o wide` option to render the node the Pod is running on in a new column:

```
$ kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
...				
kube-proxy-csrww	1/1	Running	0	4d22h
kube-proxy-fjd48	1/1	Running	0	4d22h
kube-proxy-tvf52	1/1	Running	0	4d22h

Take a look at the event log for `kube-proxy` Pods that seem to have an issue. The following command describes the Pod named `kube-proxy-csrww`. In addition, you might find more information in the event log of the corresponding DaemonSet:

```
$ kubectl describe pod kube-proxy-csrww -n kube-system
$ kubectl describe daemonset kube-proxy -n kube-system
```

The logs may come in handy as well. You will be able to check the logs for the `kube-proxy` Pod that runs on the specific worker node:

```
$ kubectl describe pod kube-proxy-csrww -n kube-system | grep Node:
Node:                               worker-1/10.0.2.15
$ kubectl logs kube-proxy-csrww -n kube-system
```

Summary

Troubleshooting Kubernetes clusters and nodes requires a deep understanding of component interactions, systematic investigation skills, and familiarity with various debugging tools. The scenarios covered in this chapter represent common real-world issues you'll encounter when managing Kubernetes infrastructure. By mastering these troubleshooting techniques, you'll be able to quickly diagnose and resolve cluster-level problems, minimizing downtime and maintaining service reliability.

Exam Essentials

Master quick node diagnostics and recovery

In the exam, you must rapidly identify why nodes are `NotReady` and fix them within minutes. Memorize this sequence: `kubectl`

get nodes, `kubectl describe node <node-name>`, check the Conditions section for specific issues (MemoryPressure, DiskPressure, PIDPressure, Ready). Know how to SSH into nodes and use `systemctl status kubelet`, `systemctl restart kubelet`, and `journalctl -u kubelet | tail -50` to diagnose and fix kubelet issues.

Know system Pod locations and recovery commands

You must instantly recognize and fix control plane component failures. Remember that static Pods (API server, controller-manager, scheduler, etcd) have their manifests in the directory `/etc/kubernetes/manifests` and their logs accessible via `kubectl logs <component>-<node-name> -n kube-system`. For crashed components, know that editing the manifest file directly will trigger the kubelet to restart the Pod automatically.

Fix Pod scheduling issues

Understand how node issues prevent Pod scheduling and know the quick fixes. When Pods are Pending, check for node taints (`kubectl describe node | grep Taint`), verify node capacity (`kubectl describe node | grep -A5 "Allocated resources"`), and look for cordoned nodes (`kubectl get nodes | grep SchedulingDisabled`). Master these recovery commands: `kubectl uncordon <node>` to enable scheduling, `kubectl taint nodes <node> <taint-key>-` to remove taints (note the minus sign), and `kubectl drain <node> --ignore-daemonsets --delete-emptydir-data` for node maintenance.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. Your team reports that applications are not scaling properly. Upon investigation, you notice one of the worker nodes is having issues and new Pods are not being scheduled there.

This exercise assumes that you run a cluster with three nodes: one control plane node and two worker nodes. The names of the worker nodes are `worker-node-1` and `worker-node-2`.

Navigate to the directory `app-a/ch22/troubleshooting-worker-node` of the checked-out GitHub repository [bmuschko/cka-study-guide](#). Transfer the file `setup.sh` to the `worker-node-2`. SSH into the host machine of the `worker-node-2` node. Execute the script. For proper execution of the script, make sure that you have `kubectl` installed on `worker-`

`node-2` beforehand and can authenticate with the API server of the control plane node.

Identify why `worker-node-2` is not accepting new Pods. Fix all issues preventing the node from functioning properly. Ensure the node can accept new Pod scheduling. Verify the fix by running a test Pod on that specific node.

2. The cluster is experiencing issues with Pod scheduling. New deployments are creating ReplicaSets, but Pods remain in `Pending` state despite having sufficient resources on nodes.

This exercise assumes that you run a cluster with at least one control plane node and one worker node. The name of the control plane node is `control-plane`.

Navigate to the directory `app-a/ch22/troubleshooting-control-plane-node` of the checked-out GitHub repository [bmuschko/cka-study-guide](https://github.com/bmuschko/cka-study-guide).

Transfer the file `setup.sh` to the `control-plane` node. SSH into the host machine of `control-plane`. Execute the script.

Create a Deployment named `test-app` with the container image `nginx:1.29.1` with three replicas.

Identify which cluster component is failing on the `control-plane` node. Diagnose the root cause of the failure. Fix the component to restore proper cluster operation. Verify that Pod scheduling is working again.